

React

1.What is JSX?

JSX = JavaScript XML

JSX is a syntax (way of writing code) used in React.

It lets you write HTML-like code inside JavaScript.

JSX looks like HTML

But it is not HTML

Browser can't understand JSX directly

Example (JSX)

```
const element = <h1>Hello World</h1>;
```

This is JSX

It looks like HTML, but it runs inside JavaScript.

Why do we use JSX?

Before JSX, React code looked like this

```
React.createElement("h1", {}, "Hello");
```

1.Hard to read

2.Hard to write

3.Messy for big UI

Now with JSX

```
<h1>Hello World</h1>
```

1.Easy

2.Clean

3.Looks like HTML

What problem does JSX solve?

Problem without JSX

- UI code becomes complex
- Mixing HTML + JS is ugly
- Hard to understand component structure

Solution with JSX

- UI becomes readable
- Easy to see UI structure
- Faster development
- Less bugs

JSX makes React UI code simple and human-friendly

How JSX actually works (important)

JSX is not understood by browser.

So JSX is converted into JavaScript using Babel.

JSX code

```
<h1>Hello</h1>
```

Converted JS

```
React.createElement("h1", null, "Hello");
```

Browser understands this JS

JSX is just developer-friendly syntax

JSX Example in React Component

```
function App() {  
  
  return (  
  
    <div>  
  
      <h1>Hello React</h1>  
  
      <p>JSX makes UI easy</p>  
  
    </div>  
  
  );  
}
```

```
export default App;
```

What's happening:

- return JSX
- JSX describes UI

- React converts it to JS
- Browser shows UI

JSX Rules (Simple)

1. JSX must have one parent

`<div><h1/></div>` (right)

`<h1/><p/>` (wrong)

2. Use `className` instead of `class`

`<div className="box"></div>`

3. JavaScript inside `{ }`

`<h1>{2 + 2}</h1>`

2. Bundler

A bundler is a tool that takes multiple JavaScript, CSS, and other files from a project and combines them into one or a few optimized files so the browser can load the application efficiently.

What does a Bundler do?

A bundler:

- Combines multiple files into a single file
- Handles import and export
- Converts JSX to JavaScript
- Converts modern JavaScript to older versions
- Optimizes code (minification)
- Improves performance

How will you get bundler in your files

1. `npm init.` then you will do all things in your terminal `npm install parcel`

2. `npm install react react-dom`

3. NPM

npm (Node Package Manager) is a tool used to install, manage, and share JavaScript packages (libraries) for Node.js and frontend projects.

All code is present there related to JS like DB.

4. Node Modules

`node_modules` is a folder where npm stores all installed packages (libraries and tools) required for a JavaScript or Node.js project.

Why does `node_modules` have so many files?

Because one package depends on many other packages, and each of those depends on more packages.

Dependency chain ban jaati hai.

5. Version = Software version number

Format:

MAJOR.MINOR.PATCH

Example:

2.16.3

- 2 → Major
- 16 → Minor
- 3 → Patch

1. Major

- Big changes
- Old code may stop working
- Old code break.
- Example: React 17 → React 18

2. Minor

- New features added
- Old code still works fine

3. Patch

- Small bug fixes
- Nothing else changes

Version	What changed?
2.0.0	Major → old code may break
2.1.0	Minor → new features added, safe
2.1.1	Patch → bugs fixed only

npm example

"parcel": "^2.16.3"

- Major = 2
- Minor = 16
- Patch = 3
- ^ carrot.
- ~ tilde only takes patches
- ^ → npm can automatically update minor & patch, but not major.

- Major → breaking
- Minor → new features
- Patch → bug fixes only

6.package-lock.json

package-lock.json is a file that locks the exact versions of all installed packages in your project.

- It ensures that every developer / every machine installs the same versions of packages.
- Helps avoid “it works on my machine but not on yours” problem.

Why we need it?

1. Exact versions
 - package.json may say: "react": "^18.2.0"
 - ^18.2.0 → could install 18.2.1 tomorrow
 - package-lock.json → locks it to 18.2.0 exactly
2. Dependencies of dependencies
 - Packages depend on other packages
 - package-lock.json records exact tree
3. Faster install
 - npm uses package-lock.json to install packages quickly

Don't push your node module bcz it takes lot of space 300 mb you will generate automatically write some commands in your terminal.

simple npm install kr dena

7.If I don't push node_modules or package-lock.json, will my project work?

Yes, your project can still work, but with conditions.

1.Why node_modules is not pushed?

- node_modules contains all installed packages (React, Parcel, etc.)
- These packages are already listed in package.json
- Anyone who clones the project can run:

npm install

- npm will download everything listed in package.json
- So project will still work

2.What about package-lock.json?

- package-lock.json locks exact versions of all packages and dependencies
- If you don't push it:
 - npm will install latest compatible versions according to package.json
 - Minor or patch updates may differ
 - Sometimes project may behave slightly differently
- If you push package-lock.json → everyone gets exact same versions, fewer bugs

3.Bottom line

File	Push or Not	Effect
node_modules	NO	OK, npm install will recreate
package-lock.json	NO	OK, but versions may differ slightly
package.json	YES	MUST push, lists all dependencies

Reason your project will still work

- npm reads package.json
- Installs required packages
- Project runs normally

Caution: Without package-lock.json, if packages update in future, your code may break due to version changes (rare, but possible).

→ Whenever you install anything using npm, it goes into your node_modules folder.

All related information about these packages is stored in package.json,

and the exact versions that are installed are recorded in package-lock.json.

This is how you know which version your project is running.